

Reconhecimento de dez dígitos em matrizes bipolares 10×10 com o algoritmo do perceptron

Resumo

Este trabalho implementa um perceptron de camada única para reconhecer os dez dígitos decimais, cada um representado por uma matriz 10×10 de pixels. A rede tem 100 entradas e 10 neurônios de saída, e é treinada pelo algoritmo clássico do perceptron com codificação bipolar. O treinamento converge em quatro épocas e a rede acerta os dez dígitos. Em seguida, avalia-se a robustez a ruído: pixels são invertidos aleatoriamente com probabilidade p e mede-se a acurácia resultante. A rede mantém mais de 90 % de acerto até $p \approx 0,22$, isto é, com cerca de 22 dos 100 pixels trocados, e só degrada ao nível do acaso quando p se aproxima de 0,5. Acompanham o relatório três demonstrações interativas usadas na apresentação.

Palavras-chave: perceptron, redes neurais, reconhecimento de dígitos, ruído.

1 Introdução

O reconhecimento de dígitos é o problema de referência para apresentar classificadores neurais. Sehta, Saraf e Vinchi [1] compararam cinco arquiteturas sobre a base MNIST — de um perceptron multicamadas até uma rede VGG16 pré-treinada — e observaram que todas ultrapassam 97 % de acurácia. A conclusão deles é que, quando o desempenho satura, a escolha do modelo passa a ser ditada pelo custo computacional: a VGG16 leva 42 min para treinar, contra 24 s do perceptron multicamadas.

Este trabalho segue o caminho inverso e pergunta qual é o *piso*: o que a rede neural mais simples possível — um perceptron de camada única, sem camada oculta e sem retropropagação — consegue fazer nessa tarefa, e como ela falha quando a imagem se degrada.

Os objetivos são três:

1. implementar o algoritmo do perceptron conforme Fausett [3] e treinar a rede nos dez dígitos;
2. verificar a convergência e visualizar os pesos aprendidos;
3. medir quanto ruído a rede tolera antes de começar a errar.

2 O perceptron

2.1 Representação dos dígitos

Cada dígito é desenhado numa matriz 10×10 (Figura 1) e achatado num vetor de 100 componentes. Adota-se a **codificação bipolar**: pixel aceso vale +1 e pixel apagado vale -1.

A escolha do bipolar em lugar do binário (0 e 1) não é detalhe. Na regra de atualização, o ajuste de cada peso é proporcional a x_i ; se o pixel apagado valesse zero, ele nunca contribuiria com nada, e a *ausência de tinta deixaria de ser informação*. Com -1, os dois estados pesam igual [3].

A saída desejada usa codificação *um-de-dez* bipolar: para o dígito k , o neurônio k deve responder +1 e todos os outros -1.

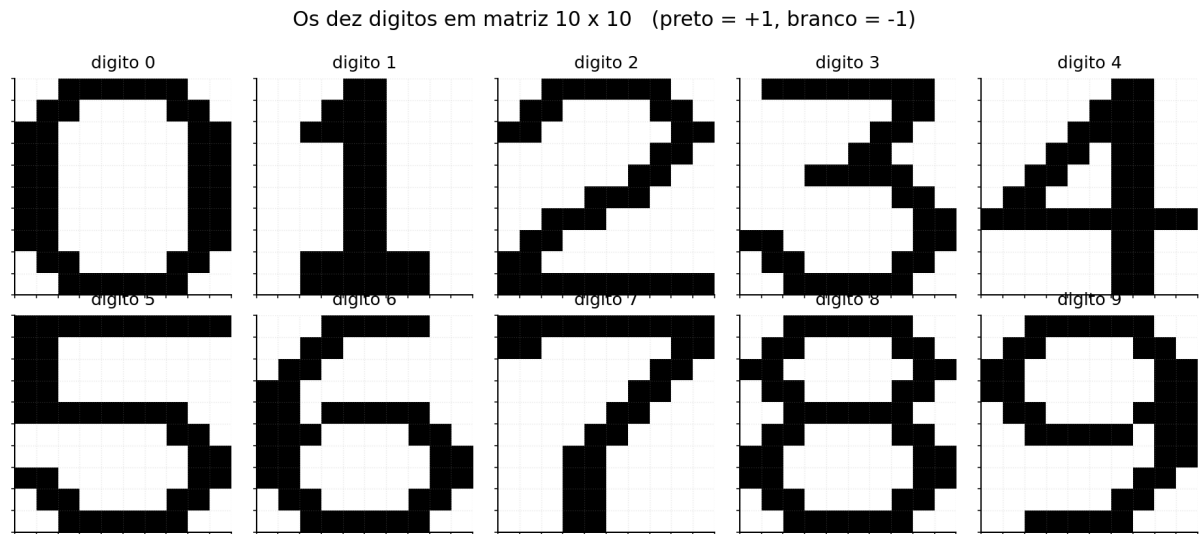


Figura 1: Os dez dígitos usados no treinamento. Cada um é uma matriz 10×10 ; preto representa +1 e branco representa -1.

2.2 Arquitetura e algoritmo

A rede tem $n = 100$ entradas e $m = 10$ neurônios de saída, num total de $100 \times 10 + 10 = 1010$ parâmetros. Cada neurônio j calcula a entrada líquida

$$y_in_j = b_j + \sum_{i=1}^{100} x_i w_{ij}, \quad (1)$$

e aplica a função degrau bipolar

$$y_j = \begin{cases} +1, & \text{se } y_in_j \geq \theta, \\ -1, & \text{caso contrário.} \end{cases} \quad (2)$$

O treinamento segue o algoritmo do perceptron [2, 3]:

- **Passo 0.** Pesos e bias começam em zero.
- **Passo 1.** Repetir enquanto algum peso mudar.
- **Passo 2–4.** Para cada dígito, calcular y_in_j e y_j .
- **Passo 5.** Se $t_j \neq y_j$, corrigir:

$$b_j \leftarrow b_j + \alpha t_j, \quad w_{ij} \leftarrow w_{ij} + \alpha t_j x_i. \quad (3)$$

Se acertou, não mexe em nada.

- **Passo 6.** Parar quando uma época inteira passar sem nenhuma correção.

Usou-se $\alpha = 1$ e $\theta = 0$. A Listagem 1 mostra o laço principal, que é a tradução direta dos Passos 2 a 5.

Listing 1: Uma época de treinamento (arquivo `perceptron.py`).

```

1 for k in range(len(X)): # Passo 2
2     x = X[k] # Passo 3
3     t = T[k]
4 
```

```

5  y_in = self.b + x @ self.W # Passo 4
6  y = self.ativacao(y_in)
7
8  erro = y != t # Passo 5: so corrige onde errou
9  if erro.any():
10     correcao = self.alfa * t * erro
11     self.W += np.outer(x, correcao)
12     self.b += correcao
13     n_atualizacoes += int(erro.sum())

```

Como a rede decide. A leitura direta da saída bipolar pode ser ambígua: sob ruído acontece de nenhum neurônio responder +1, ou de vários responderem ao mesmo tempo. Por isso o dígito predito é o do neurônio com maior y_{in} , ou seja, o mais “confiante”.

3 Resultados

3.1 Treinamento

A rede convergiu em **quatro épocas**, como mostra a Tabela 1 e a Figura 2. Ao fim da segunda época ela já acertava os dez dígitos, mas o algoritmo continuou por mais duas épocas.

Tabela 1: Evolução do treinamento

Época	Correções de peso	Dígitos corretos
1	39	7 de 10
2	10	10 de 10
3	4	10 de 10
4	0	10 de 10

Treinamento do perceptron

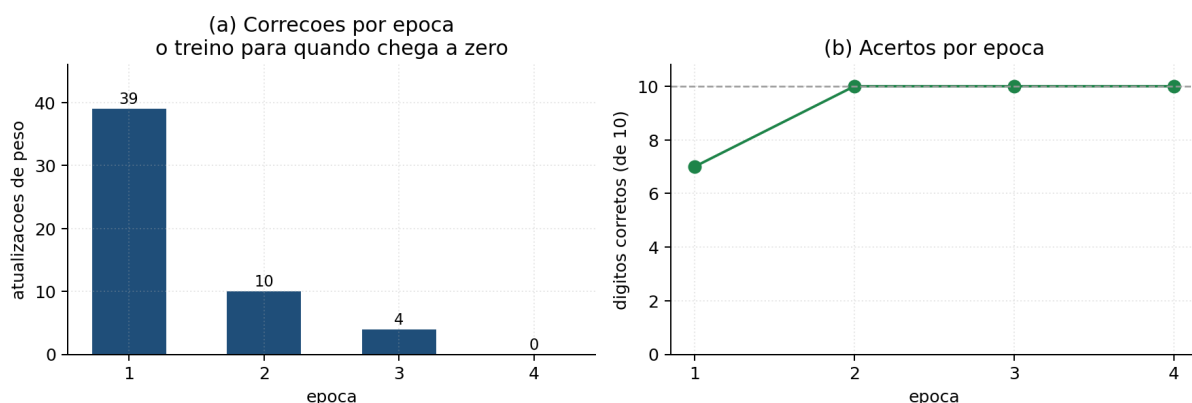


Figura 2: Treinamento do perceptron. Em (a), o número de correções cai a zero na quarta época, que é o critério de parada. Em (b), os acertos chegam a 10 de 10 já na segunda época.

Essa diferença entre a segunda e a quarta época merece explicação, porque costuma causar estranheza. Os dois critérios não são o mesmo. O acerto é medido pelo neurônio vencedor; o algoritmo, por sua vez, exige que *cada um* dos dez neurônios responda exatamente o valor esperado. Um dígito pode ser classificado corretamente e ainda assim provocar correções, porque

algum neurônio de outra classe respondeu +1 quando deveria responder -1. As épocas extras não aumentam o acerto: elas aumentam a **folga** da decisão, e é essa folga que se transforma em resistência ao ruído.

A convergência era esperada. O teorema de convergência do perceptron [4] garante que, se as classes forem linearmente separáveis, o algoritmo pára após um número finito de correções. Aqui são apenas dez pontos num espaço de 100 dimensões, bem distantes entre si, então a separabilidade é folgada.

3.2 O que a rede aprendeu

Como cada neurônio tem 100 pesos, um por pixel, esses pesos podem ser reorganizados de volta numa imagem 10×10 (Figura 3). O resultado é legível: o neurônio 1 concentra pesos positivos na coluna central, o neurônio 7 forma a diagonal descendente, o neurônio 0 desenha um anel com centro negativo.

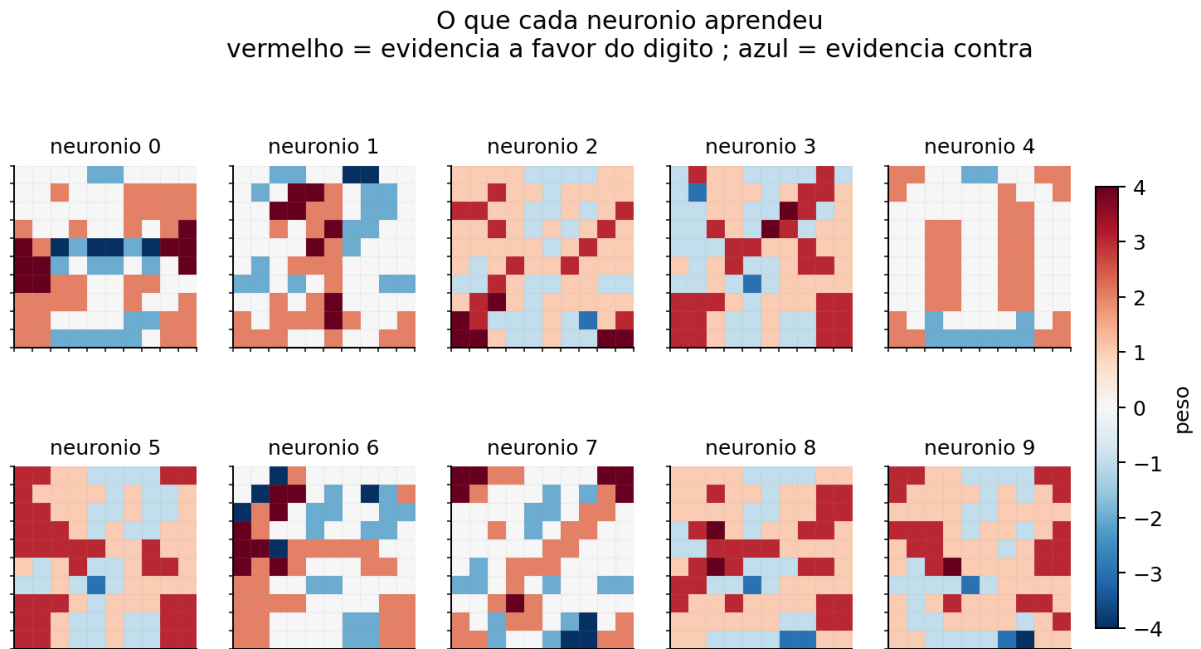


Figura 3: Pesos de cada neurônio vistos como imagem. Vermelho é peso positivo (o pixel favorece aquele dígito) e azul é peso negativo.

A razão aparece na regra (3): cada correção soma ou subtrai o próprio padrão de entrada aos pesos. O vetor de pesos acaba sendo uma combinação dos dígitos vistos no treino, funcionando como um *molde* do dígito. Essa transparência é uma vantagem didática do perceptron: nas redes profundas comparadas em [1] não há nada igualmente direto de se olhar.

3.3 Ruído

Para testar a robustez, cada pixel é invertido de forma independente com probabilidade p — o análogo bipolar do ruído sal-e-pimenta (Figura 4). Para cada valor de p foram feitos 300 sorteios dos dez dígitos, e a acurácia é a média desses 3000 testes.

Os resultados estão na Tabela 2 e na Figura 5. Três leituras:

A queda é suave, não abrupta. Com 10 % dos pixels trocados a acurácia mal se move (99.4 %), e mesmo com 20 % ela ainda é de 92.8 %. O motivo está na soma da equação (1): os

O dígito 3 com ruído crescente (p = fracão de pixels invertidos)

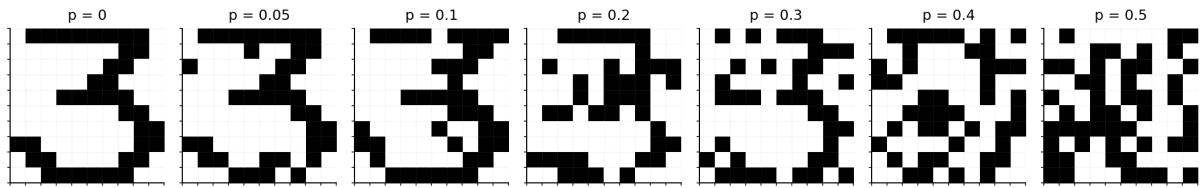


Figura 4: O dígito 3 com ruído crescente. Em $p = 0,30$ o padrão já é difícil de ler a olho nu, mas a rede ainda acerta 77 % dos casos.

Tabela 2: Acurácia em função do ruído

p	Pixels trocados (média)	Acurácia
0,00	0	100.0 %
0,10	10	99.4 %
0,20	20	92.8 %
0,25	25	86.5 %
0,30	30	77.1 %
0,40	40	44.1 %
0,50	50	9.8 %

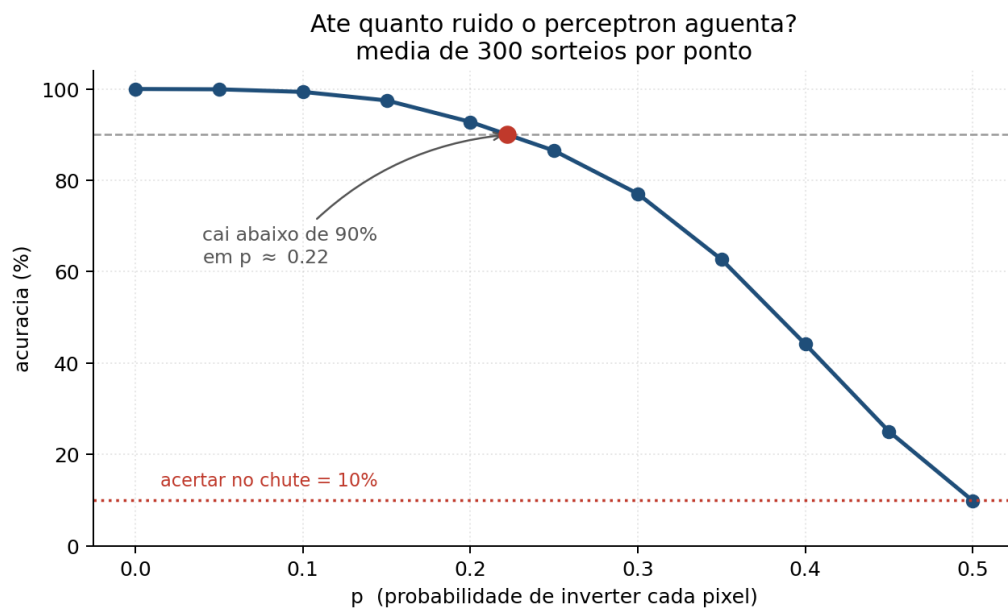


Figura 5: Acurácia em função da probabilidade de inversão de pixel. A queda é suave até cerca de $p = 0,2$ e depois acelera.

100 pixels contribuem de forma independente, então inversões aleatórias tendem a se cancelar umas às outras. A rede não depende de nenhum pixel em particular.

O limite de 90 % fica em torno de $p = 0,22$. Esse é um número prático: cerca de 22 pixels dos 100 podem estar errados sem que a taxa de acerto caia abaixo de 90 %.

Em $p = 0,5$ o desempenho é o do chute. A acurácia vai a 9.8 %, praticamente o 10 % de acertar ao acaso entre dez dígitos. Faz sentido: quando metade dos pixels é invertida ao acaso, a imagem perde toda relação com o padrão original, e não há informação alguma para a rede usar.

4 Demonstrações interativas

Para a apresentação foram preparadas três demonstrações, no arquivo `demo.py`, que abrem numa janela e podem ser manipuladas ao vivo:

1. **Treino passo a passo.** Um botão avança uma época por vez e os dez mapas de peso vão se formando na tela, partindo do zero até a convergência.
2. **Ruído com controle deslizante.** Um cursor ajusta p e os dez dígitos aparecem degradados em tempo real, com o rótulo predito em verde (acerto) ou vermelho (erro).
3. **Desenhar o dígito.** Desenha-se um dígito com o mouse numa grade 10×10 e a rede classifica a cada traço, exibindo o valor de y_{in} de cada neurônio num gráfico de barras.

A terceira é a mais reveladora em sala: um dígito desenhado à mão, diferente do padrão de treino, costuma ser reconhecido mesmo assim — mas se for deslocado um pixel para o lado, a resposta muda bastante. Isso expõe de forma concreta que o perceptron **não tem invariância a translação**, limitação que as redes convolucionais de [6] resolvem.

5 Conclusão

O perceptron de camada única, com 1010 parâmetros e quatro épocas de treinamento, reconhece os dez dígitos e tolera a inversão de cerca de 22 % dos pixels mantendo 90 % de acerto. Para efeito de comparação, o menor modelo avaliado em [1] tem 118 282 parâmetros — mais de cem vezes mais.

Isso não significa que o perceptron seja competitivo com aquelas arquiteturas. A diferença está no problema, não no algoritmo: aqui há um único exemplar ideal por classe, e dez pontos bem espaçados são trivialmente separáveis por um hiperplano. Com a variedade de caligrafias da base MNIST, cada classe vira uma nuvem de pontos que invade a das outras, e aí o perceptron de camada única falha — é exatamente a limitação que Minsky e Papert [5] formalizaram e que motivou o desenvolvimento das redes multicamadas.

Como continuação natural, seria interessante incluir vários exemplares por dígito até encontrar o ponto em que a separabilidade linear se quebra, e então comparar com uma rede de uma camada oculta sobre o mesmo conjunto.

Referências

- [1] N. Sehta, K. Saraf e A. Vinchi, “Evaluation of Classifiers Based on Neural Network for Handwritten Digit Recognition,” *2024 International Conference on Advances in Computing Research on Science Engineering and Technology (ACROSET)*, IEEE, 2024. DOI: 10.1109/ACROSET62108.2024.10743433.

- [2] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, n. 6, pp. 386–408, 1958.
- [3] L. V. Fausett, *Fundamentals of Neural Networks: Architectures, Algorithms, and Applications*. Prentice Hall, 1994.
- [4] A. B. J. Novikoff, “On convergence proofs for perceptrons,” *Proceedings of the Symposium on the Mathematical Theory of Automata*, vol. 12, pp. 615–622, 1962.
- [5] M. Minsky e S. Papert, *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.
- [6] Y. LeCun, L. Bottou, Y. Bengio e P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, n. 11, pp. 2278–2324, 1998.

Apêndice A — Código-fonte: perceptron.py

Arquivo principal do trabalho. Contém os dez padrões, o algoritmo do perceptron (Passos 0 a 6) e o gerador de ruído. Executando `python3 perceptron.py` obtêm-se o treinamento completo e a tabela de ruído reproduzidos na Seção 3.

```
1  """
2  Perceptron para reconhecimento de 10 dígitos em matriz 10x10.
3  Trabalho 04 - Disciplina de Redes Neurais
4
5  Este arquivo contém TUDO o que a rede precisa: os padrões, o algoritmo de
6  treinamento e o gerador de ruído. São pouco mais de 200 linhas, pensadas
7  para serem lidas de cima a baixo durante a apresentação.
8
9  Convenção bipolar: pixel aceso = +1 | pixel apagado = -1
10  Arquitetura: 100 entradas (10x10) -> 10 neurônios de saída
11  """
12
13  import numpy as np
14
15  LADO = 10 # a matriz é 10x10
16  N_ENTRADAS = LADO * LADO # 100 pixels
17  N_SAIDAS = 10 # um neurônio por dígito
18
19
20  # =====
21  # 1. OS DEZ DÍGITOS
22  # =====
23  # '#' = pixel aceso (+1) '.' = pixel apagado (-1)
24
25  DÍGITOS = {
26      0: [".#####.",
27         ".##....##.",
28         "##.....##",
29         "##.....##",
30         "##.....##",
31         "##.....##",
32         "##.....##",
33         "##.....##",
34         ".##....##.",
35         "#####."],
36
37      1: ["....##....",
38         "...##....",
39         ".###....",
40         "...##....",
41         "...##....",
42         "...##....",
43         "...##....",
44         "...##....",
45         ".#####.",
46         "#####."],
47
48      2: [".#####.",
49         ".##....##.",
50         "##.....##",
51         ".....##.",
52         ".....##.",
53         ".....##.",
54         ".###....",
55         ".##.....",
56         "##.....",
57         "#####."],
58
59      3: [".#####.",
60         ".....##.",
61         ".....##.",
62         ".....##.",
63         ".....##.",
64         ".....##.",
65         ".....##.",
66         "##.....##",
```

```

67         ".##....##.",
68         "..#####."],
69
70     4: ["......##.",
71         ".....###.",
72         ".....#####.",
73         "...##.##.",
74         "...##.##.",
75         ".##....##.",
76         "#####.",
77         ".....##.",
78         ".....##.",
79         ".....##."],
80
81     5: ["#####.",
82         "##.....",
83         "##.....",
84         "##.....",
85         "#####.",
86         ".....##.",
87         ".....##.",
88         "##.....##.",
89         ".##....##.",
90         "..#####."],
91
92     6: [".#####.",
93         ".##.....",
94         ".##.....",
95         "##.....",
96         "##.#####.",
97         "###....##.",
98         "##.....##.",
99         "##.....##.",
100        ".##....##.",
101        "..#####."],
102
103     7: ["#####.",
104         "##.....##.",
105         ".....##.",
106         ".....##.",
107         ".....##.",
108         ".....##.",
109         "...##.....",
110         "...##.....",
111         "...##.....",
112         "...##....."],
113
114     8: [".#####.",
115         ".##....##.",
116         "##.....##.",
117         ".##....##.",
118         ".#####.",
119         ".##....##.",
120         "##.....##.",
121         "##.....##.",
122         ".##....##.",
123         "..#####."],
124
125     9: [".#####.",
126         ".##....##.",
127         "##.....##.",
128         "##.....##.",
129         ".##....##.",
130         "...#####.",
131         ".....##.",
132         ".....##.",
133         ".....##.",
134         "..#####."],
135 }
136
137
138 def carregar_digitos():
139     """

```

```

140     Monta o conjunto de treinamento.
141
142     Retorna:
143     X (10, 100) cada linha e um dígito achatado em vetor bipolar
144     T (10, 10) alvos: T[k] tem +1 na posição k e -1 nas outras
145     y (10,) o rótulo de cada linha: [0, 1, 2, ..., 9]
146     """
147     X = np.array([
148         [1.0 if c == "#" else -1.0 for linha in DIGITOS[d] for c in linha]
149         for d in range(10)
150     ])
151     T = -np.ones((10, 10))
152     T[np.arange(10), np.arange(10)] = 1.0 # diagonal = +1
153     return X, T, np.arange(10)
154
155
156 def para_imagem(vetor):
157     """Transforma o vetor de 100 componentes de volta na matriz 10x10."""
158     return np.asarray(vetor, dtype=float).reshape(LADO, LADO)
159
160
161 # =====
162 # 2. O PERCEPTRON
163 # =====
164
165 class Perceptron:
166     """
167     Perceptron de camada única, com o algoritmo clássico do Fausett.
168
169     Passo 0. Inicializa pesos e bias com zero.
170     Passo 1. Repete enquanto houver mudança de peso.
171     Passo 2. Para cada par de treinamento (s, t):
172     Passo 3. x = s
173     Passo 4. y_in_j = b_j + soma_i (x_i * w_ij)
174             y_j = +1 se y_in_j >= 0, senão -1
175     Passo 5. Se t_j != y_j:
176             b_j += alfa * t_j
177             w_ij += alfa * t_j * x_i
178     Passo 6. Para quando nenhum peso mudou na época.
179     """
180
181     def __init__(self, alfa=1.0, limiar=0.0):
182         self.alfa = alfa # taxa de aprendizado
183         self.limiar = limiar # theta da função degrau
184         self.W = np.zeros((N_ENTRADAS, N_SAIDAS)) # Passo 0
185         self.b = np.zeros(N_SAIDAS)
186         self.historico = [] # guarda o que aconteceu em cada época
187
188     # -- Passo 4 -----
189     def entrada_liquida(self, X):
190         """y_in = b + X.W (aceita um dígito ou vários de uma vez)."""
191         return np.atleast_2d(X) @ self.W + self.b
192
193     def ativacao(self, y_in):
194         """Função degrau bipolar: +1 se y_in >= limiar, senão -1."""
195         return np.where(y_in >= self.limiar, 1.0, -1.0)
196
197     def classificar(self, X):
198         """
199         Qual dígito a rede acha que é?
200
201         A saída bipolar pode ser ambígua (nenhum ou vários neurônios em +1),
202         então decidimos pelo neurônio de maior y_in -- o "mais confiante".
203         """
204         return np.argmax(self.entrada_liquida(X), axis=1)
205
206     # -- Passos 1 a 6 -----
207     def treinar_uma_epoca(self, X, T):
208         """
209         Executa UMA época (uma passada por todos os padrões).
210
211         Retorna quantas atualizações de peso foram feitas. Zero significa
212         que a rede convergiu.

```

```

213     """
214     n_atualizacoes = 0
215
216     for k in range(len(X)): # Passo 2
217         x = X[k] # Passo 3
218         t = T[k]
219
220         y_in = self.b + x @ self.W # Passo 4
221         y = self.ativacao(y_in)
222
223         erro = y != t # Passo 5: so corrige onde erro
224         if erro.any():
225             correcao = self.alfa * t * erro
226             self.W += np.outer(x, correcao)
227             self.b += correcao
228             n_atualizacoes += int(erro.sum())
229
230     return n_atualizacoes
231
232 def treinar(self, X, T, max_epocas=100, mostrar=True):
233     """Repete epocas ate convergir (Passo 6) ou atingir max_epocas."""
234     rotulos = np.argmax(T, axis=1)
235
236     if mostrar:
237         print(f"{'epoca':>6} | {'atualizacoes':>13} | {'acertos':>8}")
238         print("-" * 34)
239
240     for epoca in range(1, max_epocas + 1):
241         n = self.treinar_uma_epoca(X, T)
242         acertos = int((self.classificar(X) == rotulos).sum())
243         self.historico.append((epoca, n, acertos))
244
245         if mostrar:
246             print(f"{'epoca':>6} | {'n':>13} | {'acertos':>6}/10")
247
248         if n == 0: # Passo 6
249             if mostrar:
250                 print("-" * 34)
251                 print(f"Convergiu na epoca {epoca}: nenhum peso mudou.")
252             return epoca
253
254         if mostrar:
255             print(f"Parou no limite de {max_epocas} epocas.")
256     return max_epocas
257
258 # -- utilidades -----
259 def acuracia(self, X, rotulos):
260     """Fracao de acertos (0 a 1)."""
261     return float((self.classificar(X) == rotulos).mean())
262
263 def mapa_de_pesos(self, j):
264     """Os pesos do neuronio j vistos como imagem 10x10."""
265     return self.W[:, j].reshape(LADO, LADO)
266
267
268 # =====
269 # 3. RUIDO
270 # =====
271
272 def adicionar_ruido(X, p, rng=None):
273     """
274     Inverte cada pixel com probabilidade p (ruído sal-e-pimenta).
275
276     p = 0.0 -> nada muda
277     p = 0.2 -> em media 20 dos 100 pixels sao invertidos
278     p = 0.5 -> a imagem vira ruido puro, sem relacao com o original
279     """
280     rng = np.random.default_rng() if rng is None else rng
281     X = np.array(X, dtype=float) # copia, nao altera o original
282     inverter = rng.random(X.shape) < p
283     X[inverter] *= -1
284     return X
285

```

```

286
287 def testar_com_ruido(rede, X, rotulos, p, repeticoes=200, rng=None):
288     """
289     Mede a acuracia da rede em um nivel de ruido p.
290
291     Como o ruido e aleatorio, uma unica medida nao diz muito: repetimos
292     varias vezes e tiramos a media.
293     """
294     rng = np.random.default_rng(0) if rng is None else rng
295     acertos = 0
296     for _ in range(repeticoes):
297         acertos += int((rede.classificar(adicionar_ruido(X, p, rng))
298                        == rotulos).sum())
299     return acertos / (repeticoes * len(X))
300
301
302 # =====
303 # 4. EXECUCAO PRINCIPAL
304 # =====
305
306 def main():
307     print("=" * 60)
308     print("PERCEPTRON - RECONHECIMENTO DE 10 DIGITOS (matriz 10x10)")
309     print("=" * 60)
310
311     X, T, rotulos = carregar_digitos()
312     print(f"\nConjunto de treinamento: {X.shape[0]} digitos, "
313           f"{X.shape[1]} pixels cada.\n")
314
315     # ---- treinamento ----
316     print("TREINAMENTO")
317     rede = Perceptron(alfa=1.0, limiar=0.0)
318     epocas = rede.treinar(X, T)
319     print(f"\nAcuracia nos digitos originais: "
320           f"{rede.acuracia(X, rotulos):.0%}\n")
321
322     # ---- o que a rede responde para cada digito ----
323     print("SAIDA DA REDE PARA CADA DIGITO")
324     print(f"{'digito':>7} | {'predito':>8} | {'y_in do vencedor':>17}")
325     print("-" * 40)
326     y_in = rede.entrada_liquida(X)
327     for k in rotulos:
328         print(f"{k:>7} | {rede.classificar(X[k])[0]:>8} | {y_in[k].max():>17.0f}")
329
330     # ---- analise de ruido ----
331     print("\nANALISE DE RUÍDO")
332     print("Quantos pixels podem ser invertidos antes da rede errar?\n")
333     print(f"{'p':>6} | {'pixels trocados':>16} | {'acuracia':>9}")
334     print("-" * 38)
335     for p in [0.0, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.40, 0.50]:
336         acc = testar_com_ruido(rede, X, rotulos, p)
337         print(f"{p:>6.2f} | {int(p * 100):>14} % | {acc:>8.1%}")
338
339     print("\nObs.: em p = 0.50 a imagem perde toda relacao com o original,")
340     print("entao acertar 10% e o mesmo que chutar entre 10 digitos.")
341     print("\nPara as demonstracoes interativas, rode: python3 demo.py")
342
343
344 if __name__ == "__main__":
345     main()

```

Apêndice B — Código-fonte: demo.py

As três demonstrações interativas descritas na Seção 4. Cada uma abre uma janela do matplotlib e pode ser executada isoladamente com `python3 demo.py 1, 2` ou `3`.

```
1 """
2 Demonstrações interativas para a apresentação.
3 Trabalho 04 - Disciplina de Redes Neurais
4
5 Rode: python3 demo.py (mostra o menu)
6       python3 demo.py 1 (treino passo a passo)
7       python3 demo.py 2 (ruído com controle deslizante)
8       python3 demo.py 3 (desenhar o dígito com o mouse)
9
10 As três demos abrem uma janela do matplotlib. Não precisa de Jupyter.
11 """
12
13 import sys
14
15 import numpy as np
16 import matplotlib.pyplot as plt
17 from matplotlib.widgets import Button, Slider
18
19 from perceptron import (LADO, Perceptron, adicionar_ruído, carregar_dígitos,
20                          para_imagem)
21
22 # cores usadas nas três demos
23 VERDE = "#1e8449"
24 VERMELHO = "#c0392b"
25 AZUL = "#1f4e79"
26 CINZA = "#7f8c8d"
27 LARANJA = "#b9770e"
28
29
30 def _desenhar_grade(ax, matriz, cmap="gray_r", vmin=-1, vmax=1):
31     """Desenha uma matriz 10x10 com a gradinha por cima."""
32     im = ax.imshow(matriz, cmap=cmap, vmin=vmin, vmax=vmax,
33                    interpolation="nearest")
34     ax.set_xticks(np.arange(-0.5, LADO, 1), minor=True)
35     ax.set_yticks(np.arange(-0.5, LADO, 1), minor=True)
36     ax.grid(which="minor", color="#999999", linewidth=0.6)
37     ax.set_xticks([])
38     ax.set_yticks([])
39     return im
40
41
42 # =====
43 # DEMO 1 - TREINO PASSO A PASSO
44 # =====
45 def demo_treino():
46     """
47     Mostra a rede aprendendo, uma época por vez.
48
49     Cada clique em "Proxima época" executa uma passada completa pelos 10
50     dígitos. Os dez mapas de peso vão se formando na tela, e o painel de
51     texto mostra quantas correções foram feitas.
52     """
53     X, T, rotulos = carregar_dígitos()
54     rede = Perceptron()
55     estado = {"epoca": 0, "convergiu": False}
56
57     fig = plt.figure(figsize=(13, 6.2))
58     fig.canvas.manager.set_window_title("Demo 1 - Treino passo a passo")
59     eixos = [fig.add_subplot(2, 5, j + 1) for j in range(10)]
60     fig.subplots_adjust(left=0.03, right=0.97, top=0.82, bottom=0.16,
61                        hspace=0.30, wspace=0.15)
62
63     imagens = []
64     for j, ax in enumerate(eixos):
65         im = _desenhar_grade(ax, rede.mapa_de_pesos(j), cmap="RdBu_r",
66                              vmin=-1, vmax=1)
67         ax.set_title(f"neurônio {j}", fontsize=9)
```

```

68     imagens.append(im)
69
70     titulo = fig.suptitle("", fontsize=12)
71
72     def atualizar_titulo(n_atualizacoes=None):
73         if estado["epoca"] == 0:
74             txt = ("Epoca 0 -- todos os pesos comecam em ZERO (Passo 0)\n"
75                   "Clique em 'Proxima epoca' para a rede aprender")
76         elif estado["convergiu"]:
77             acc = rede.acuracia(X, rotulos)
78             txt = (f"CONVERGIU na epoca {estado['epoca']} -- "
79                   f"nenhum peso mudou (Passo 6)\n"
80                   f"Acuracia: {acc:.0%} | o treinamento acabou")
81         else:
82             acertos = int((rede.classificar(X) == rotulos).sum())
83             txt = (f"Epoca {estado['epoca']}: {n_atualizacoes} correcoes de peso"
84                   f" | acertos: {acertos}/10\n"
85                   f"vermelho = peso positivo (evidencia a favor) ; "
86                   f"azul = peso negativo (contra)")
87         titulo.set_text(txt)
88
89     atualizar_titulo()
90
91     def proxima_epoca(evento):
92         if estado["convergiu"]:
93             return
94         n = rede.treinar_uma_epoca(X, T)
95         estado["epoca"] += 1
96         if n == 0:
97             estado["convergiu"] = True
98
99         # reescala as cores conforme os pesos crescem. Usamos o percentil 98
100         # em vez do maximo: uns poucos pesos extremos achatariam o contraste
101         # e esconderiam o desenho que se forma nos mapas.
102         vmax = max(1.0, float(np.percentile(np.abs(rede.W), 98)))
103         for j, im in enumerate(imagens):
104             im.set_data(rede.mapa_de_pesos(j))
105             im.set_clim(-vmax, vmax)
106         atualizar_titulo(n)
107         fig.canvas.draw_idle()
108
109     def reiniciar(evento):
110         rede.W[:] = 0.0
111         rede.b[:] = 0.0
112         rede.historico.clear()
113         estado["epoca"] = 0
114         estado["convergiu"] = False
115         for j, im in enumerate(imagens):
116             im.set_data(rede.mapa_de_pesos(j))
117             im.set_clim(-1, 1)
118         atualizar_titulo()
119         fig.canvas.draw_idle()
120
121     b1 = Button(fig.add_axes([0.34, 0.04, 0.18, 0.07]), "Proxima epoca")
122     b1.on_clicked(proxima_epoca)
123     b2 = Button(fig.add_axes([0.54, 0.04, 0.12, 0.07]), "Reiniciar")
124     b2.on_clicked(reiniciar)
125
126     # o matplotlib nao guarda referencia para os widgets: se elas sumirem,
127     # os botoes param de responder. Penduramos na figura para mante-las vivas.
128     fig.widgets = [b1, b2]
129
130     plt.show()
131     return fig
132
133
134     # =====
135     # DEMO 2 - RUÍDO COM CONTROLE DESLIZANTE
136     # =====
137     def demo_ruido():
138         """
139         Aumenta o ruido ao vivo e mostra a rede acertando ate quebrar.
140

```

```

141     0 slider controla p, a probabilidade de cada pixel ser invertido.
142     Os dez dígitos aparecem degradados, com o rótulo predito em verde
143     (acerto) ou vermelho (erro).
144     """
145     X, T, rotulos = carregar_digitos()
146     rede = Perceptron()
147     rede.treinar(X, T, mostrar=False)
148     estado = {"semente": 0}
149
150     fig = plt.figure(figsize=(13, 6.4))
151     fig.canvas.manager.set_window_title("Demo 2 - Ruído")
152     eixos = [fig.add_subplot(2, 5, k + 1) for k in range(10)]
153     fig.subplots_adjust(left=0.03, right=0.97, top=0.84, bottom=0.20,
154                        hspace=0.35, wspace=0.15)
155
156     imagens = [_desenhar_grade(ax, para_imagem(X[k])) for k, ax in enumerate(eixos)]
157     titulo = fig.suptitle("", fontsize=12)
158
159     def atualizar(_=None):
160         p = slider.val
161         rng = np.random.default_rng(estado["semente"])
162         Xr = adicionar_ruído(X, p, rng)
163         preditos = rede.classificar(Xr)
164         acertos = int((preditos == rotulos).sum())
165
166         for k, (ax, im) in enumerate(zip(eixos, imagens)):
167             im.set_data(para_imagem(Xr[k]))
168             ok = preditos[k] == rotulos[k]
169             ax.set_title(f"{rotulos[k]} -> {preditos[k]}",
170                        color=VERDE if ok else VERMELHO, fontsize=11)
171
172             titulo.set_text(
173                 f"p = {p:.2f} -> em média {int(p * 100)} dos 100 pixels "
174                 f"invertidos | acertos: {acertos}/10")
175             fig.canvas.draw_idle()
176
177     slider = Slider(fig.add_axes([0.20, 0.08, 0.60, 0.04]),
178                    "ruído p", 0.0, 0.5, valinit=0.0, valstep=0.01,
179                    color=AZUL)
180     slider.on_changed(atualizar)
181
182     def sortear(evento):
183         estado["semente"] += 1
184         atualizar()
185
186     botao = Button(fig.add_axes([0.84, 0.06, 0.12, 0.07]), "Outro sorteio")
187     botao.on_clicked(sortear)
188
189     # mantém os widgets vivos (ver comentário na demo 1)
190     fig._widgets = [slider, botao]
191
192     atualizar()
193     plt.show()
194     return fig
195
196
197 # =====
198 # DEMO 3 - DESENHAR O DÍGITO COM O MOUSE (e ensinar a rede)
199 # =====
200 def demo_desenhar():
201     """
202     Desenhe um dígito e veja a rede classificar na hora.
203
204     Mouse:
205         botão esquerdo pinta o pixel
206         botão direito apaga o pixel
207
208     Teclado:
209         0 a 9 ensina a rede: "o que eu desenhei e este dígito".
210         A rede aplica a regra de aprendizado repetidas vezes,
211         até acertar o seu desenho.
212         r reseta a rede (volta a treinar só nos 10 padrões originais)
213         c limpa a tela

```

```

214
215 O grafico de barras mostra o y_in de cada neuronio. A linha do zero
216 importa: um neuronio so "reconhece" quando seu y_in fica ACIMA dela.
217 Se todas as barras estiverem abaixo de zero, nenhum neuronio reconheceu
218 o desenho -- e a rede nao tem resposta de verdade para dar.
219 """
220 X, T, rotulos = carregar_digitos()
221 rede = Perceptron()
222 rede.treinar(X, T, mostrar=False)
223
224 tela = -np.ones((LADO, LADO))
225 estado = {"pintando": None, "mensagem": "", "cor_msg": "black"}
226
227 fig = plt.figure(figsize=(12.5, 6.8))
228 fig.canvas.manager.set_window_title("Demo 3 - Desenhe e ensine a rede")
229 ax_desenho = fig.add_axes([0.05, 0.28, 0.40, 0.56])
230 ax_barras = fig.add_axes([0.55, 0.28, 0.42, 0.56])
231
232 im = _desenhar_grade(ax_desenho, tela)
233 ax_desenho.set_title("Desenhe aqui\n"
234                      "esquerdo pinta | direito apaga", fontsize=10)
235
236 barras = ax_barras.bar(range(10), np.zeros(10), color=CINZA)
237 ax_barras.set_xticks(range(10))
238 ax_barras.set_xlabel("neuronio (digito)")
239 ax_barras.set_ylabel("entrada liquida y_in")
240 ax_barras.axhline(0, color="black", lw=1.2)
241 ax_barras.grid(axis="y", alpha=0.3, linestyle=":")
242 ax_barras.set_title("acima de zero = o neuronio reconheceu", fontsize=9)
243
244 titulo = fig.suptitle("", fontsize=13)
245 rodape = fig.text(0.5, 0.15, "", ha="center", fontsize=10, color="#444444")
246 ajuda = fig.text(
247     0.5, 0.035,
248     "teclas: 0-9 ensina a rede o digito que voce desenhou | "
249     "r reseta a rede | c limpa a tela",
250     ha="center", fontsize=9.5, color="#666666")
251
252 # -- atualizacao da tela -----
253 def atualizar():
254     y_in = rede.entrada_liquida(tela.reshape(-1))[0]
255     vencedor = int(np.argmax(y_in))
256     acesos = int((tela > 0).sum())
257     # quantos neuronios de fato dispararam (+1)
258     ativos = int((y_in >= rede.limiar).sum())
259
260     for j, barra in enumerate(barras):
261         barra.set_height(y_in[j])
262         if j == vencedor and ativos >= 1:
263             barra.set_color(VERDE)
264         elif y_in[j] >= rede.limiar:
265             barra.set_color("#a9cce3")
266         else:
267             barra.set_color(CINZA)
268
269     ax_barras.relim()
270     ax_barras.autoscale_view()
271
272     ordenado = np.sort(y_in)
273     margem = ordenado[-1] - ordenado[-2]
274
275     # ---- a parte importante: so afirmar quando ha o que afirmar ----
276     if acesos == 0:
277         titulo.set_text("Tela vazia -- desenhe alguma coisa")
278         titulo.set_color(CINZA)
279         rodape.set_text("")
280     elif ativos == 0:
281         # nenhuma barra passou do zero: a rede nao reconheceu nada.
282         titulo.set_text("Nenhum neuronio reconheceu este desenho")
283         titulo.set_color(LARANJA)
284         rodape.set_text(
285             f"todos os y_in estao abaixo de zero. O maior deles e o do "
286             f"neuronio {vencedor} ({y_in[vencedor]:.0f}), mas isso e so o "

```

```

287         f"'menos ruim' -- nao e um reconhecimento.\n"
288         f"Aperte a tecla do digito correto para ENSINAR a rede.")
289     elif ativos > 1:
290         empatados = [j for j in range(10) if y_in[j] >= rede.limiar]
291         titulo.set_text(f"Resposta ambigua: {ativos} neuronios ")
292         f"reconheceram {empatados}")
293         titulo.set_color(LARANJA)
294         rodape.set_text(
295             f"o mais forte e o {vencedor} (y_in = {y_in[vencedor]:.0f}), "
296             f"com margem de apenas {margem:.0f} para o segundo.\n"
297             f"Aperte a tecla do digito correto para ENSINAR a rede.")
298     else:
299         titulo.set_text(f"A rede diz: {vencedor}")
300         titulo.set_color(VERDE)
301         rodape.set_text(
302             f"y_in = {y_in[vencedor]:.0f} | margem para o segundo = "
303             f"{margem:.0f} | {acesos} pixels acesos")
304
305     if estado["mensagem"]:
306         rodape.set_text(estado["mensagem"])
307         rodape.set_color(estado["cor_msg"])
308     else:
309         rodape.set_color("#444444")
310
311     im.set_data(tela)
312     fig.canvas.draw_idle()
313
314     # -- ENSINAR: aplica a regra do perceptron ate acertar -----
315     def ensinar(digito, max_tentativas=100):
316         """
317         Treina a rede no desenho atual, repetindo a regra de correcao
318         ate que a saida bipolar fique exatamente igual ao alvo.
319
320         E o mesmo Passo 5 do algoritmo, so que aplicado a um unico
321         padrao -- o que voce acabou de desenhar.
322         """
323         if (tela > 0).sum() == 0:
324             estado["mensagem"] = "Tela vazia: desenhie algo antes de ensinar."
325             estado["cor_msg"] = LARANJA
326             atualizar()
327             return
328
329         x = tela.reshape(-1)
330         alvo = -np.ones(10)
331         alvo[digito] = 1.0
332
333         correcoes = 0
334         for tentativa in range(1, max_tentativas + 1):
335             y_in = rede.b + x @ rede.W
336             y = rede.ativacao(y_in)
337             errou = y != alvo
338             if not errou.any():
339                 break # acertou: nada mais a corrigir
340             ajuste = rede.alfa * alvo * errou
341             rede.W += np.outer(x, ajuste)
342             rede.b += ajuste
343             correcoes += int(errou.sum())
344         else:
345             tentativa = max_tentativas
346
347         # a rede aprendeu o desenho -- mas sera que esqueceu os originais?
348         ainda_certos = int((rede.classificar(X) == rotulos).sum())
349         predito = int(rede.classificar(x)[0])
350
351         if ainda_certos == 10:
352             estado["cor_msg"] = VERDE
353             aviso = "e continua acertando os 10 digitos originais."
354         else:
355             estado["cor_msg"] = LARANJA
356             aviso = (f"MAS agora so acerta {ainda_certos}/10 dos digitos "
357                     f"originais -- aprender um padrao novo mexeu nos pesos "
358                     f"dos outros. Aperte 'r' para resetar a rede.")
359

```

```

360     estado["mensagem"] = (
361         f"Ensinei que isto e um {digito}: aprendeu em {tentativa} "
362         f"passada(s), {correcoes} correcoes de peso. Agora responde "
363         f"{predito}.\n{aviso}")
364     atualizar()
365
366 def resetar_rede():
367     rede.W[:] = 0.0
368     rede.b[:] = 0.0
369     rede.historico.clear()
370     rede.treinar(X, T, mostrar=False)
371     estado["mensagem"] = ("Rede resetada: treinada de novo apenas nos 10 "
372         "digitos originais.")
373     estado["cor_msg"] = AZUL
374     atualizar()
375
376 # -- eventos do mouse -----
377 def pixel_do_evento(evento):
378     if evento.inaxes is not ax_desenho:
379         return None
380     if evento.xdata is None or evento.ydata is None:
381         return None
382     col = int(round(evento.xdata))
383     lin = int(round(evento.ydata))
384     if 0 <= lin < LADO and 0 <= col < LADO:
385         return lin, col
386     return None
387
388 def aplicar(evento):
389     pos = pixel_do_evento(evento)
390     if pos is None or estado["pintando"] is None:
391         return
392     if tela[pos] == estado["pintando"]:
393         return # ja esta assim, evita redesenhar
394     tela[pos] = estado["pintando"]
395     estado["mensagem"] = "" # desenhar limpa o aviso anterior
396     atualizar()
397
398 def ao_preSSIONar(evento):
399     if evento.button == 1:
400         estado["pintando"] = 1.0
401     elif evento.button == 3:
402         estado["pintando"] = -1.0
403     else:
404         return
405     aplicar(evento)
406
407 def ao_arrastar(evento):
408     if estado["pintando"] is not None:
409         aplicar(evento)
410
411 def ao_soltar(evento):
412     estado["pintando"] = None
413
414 def ao_teclar(evento):
415     if evento.key is None:
416         return
417     if evento.key in "0123456789":
418         ensinar(int(evento.key))
419     elif evento.key == "r":
420         resetar_rede()
421     elif evento.key == "c":
422         limpar(None)
423
424 fig.canvas.mpl_connect("button_press_event", ao_preSSIONar)
425 fig.canvas.mpl_connect("motion_notify_event", ao_arrastar)
426 fig.canvas.mpl_connect("button_release_event", ao_soltar)
427 fig.canvas.mpl_connect("key_press_event", ao_teclar)
428
429 # -- botoes -----
430 def limpar(evento):
431     tela[:] = -1.0
432     estado["mensagem"] = ""

```

```

433     atualizar()
434
435     def carregar_exemplo(evento):
436         d = int(np.random.default_rng().integers(0, 10))
437         tela[:] = para_imagem(X[d])
438         estado["mensagem"] = ""
439         atualizar()
440
441     def botao_resetar(evento):
442         resetar_rede()
443
444     b1 = Button(fig.add_axes([0.08, 0.06, 0.13, 0.07]), "Limpar (c)")
445     b1.on_clicked(limpar)
446     b2 = Button(fig.add_axes([0.24, 0.06, 0.20, 0.07]), "Carregar exemplo")
447     b2.on_clicked(carregar_exemplo)
448     b3 = Button(fig.add_axes([0.47, 0.06, 0.18, 0.07]), "Resetar rede (r)")
449     b3.on_clicked(botao_resetar)
450
451     # mantem os widgets vivos (ver comentario na demo 1)
452     fig._widgets = [b1, b2, b3]
453     fig.ensinar = ensinar # exposto para os testes
454
455     atualizar()
456     plt.show()
457     return fig
458
459
460     # =====
461     # MENU
462     # =====
463     DEMOS = {
464         "1": ("Treino passo a passo (ver os pesos se formando)", demo_treino),
465         "2": ("Ruido com controle deslizante", demo_ruido),
466         "3": ("Desenhar um dígito e ENSINAR a rede", demo_desenhar),
467     }
468
469
470     def main():
471         if len(sys.argv) > 1 and sys.argv[1] in DEMOS:
472             DEMOS[sys.argv[1]][1]()
473             return
474
475         print("=" * 56)
476         print("DEMONSTRACOES INTERATIVAS - PERCEPTRON")
477         print("=" * 56)
478         for chave, (descricao, _) in DEMOS.items():
479             print(f" {chave}. {descricao}")
480         print(" q. sair")
481
482         while True:
483             escolha = input("\nEscolha uma demo: ").strip().lower()
484             if escolha in ("q", "sair", ""):
485                 break
486             if escolha in DEMOS:
487                 print(f"Abrindo: {DEMOS[escolha][0]}")
488                 print("(feche a janela para voltar ao menu)")
489                 DEMOS[escolha][1]()
490             else:
491                 print("Opcao invalida.")
492
493
494     if __name__ == "__main__":
495         main()

```
